

OFFLINE SECURE FACIAL RECOGNITION & LIVENESS DETECTION

Secure Mobile-Based Authentication for Remote Locations & Zero-
Network Environments

PROJECT NAME

Edge AI Suite — Offline Core

EVENT PHASE

Hackathon 7.0 / NHA1

PLATFORM ECOSYSTEM

Datalake 3.0 Integration

Problem Statement & Connectivity Gap



Zero-Network Zones

Cloud-based face APIs fail completely in mining, rural, and disaster zones where data infrastructure is non-existent.



Authentication Fraud

Manual logging systems allow proxy attendance and "buddy punching," leading to significant financial loss and safety risks.



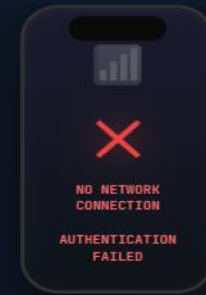
Privacy & Compliance Risks

Uploading raw biometric data to the cloud violates strict data residency laws and exposes personnel to potential identity theft.



Network Latency

Unstable edge connectivity causes delays exceeding 10 seconds, creating bottlenecks in mission-critical workforce operations.



NO SIGNAL
ZONES



BROKEN
CONNECTIVITY



TIME THEFT
& FRAUD

CONNECTIVITY GAP IMPACT

SYSTEM AVAILABILITY

100% Service Outage

OPERATIONAL INTEGRITY

Financial & Safety Risk

REGULATORY EXPOSURE

Legal & Security Liability

WORKFORCE VELOCITY

Critical Productivity Bottleneck

Target Question: How can we securely authenticate field personnel on mid-range devices without any active internet connection?

Solution Overview & Key KPIs

<1.0s

Total Auth Latency

End-to-end recognition + liveness detection optimized for mid-range mobile hardware without network overhead.

END-TO-END SYSTEM PIPELINE



A 100% offline React Native framework designed for high-security remote environments. By performing full biometric inference on-device, we eliminate the connectivity gap while ensuring absolute data privacy.

READY

100% Open Source (OSS Stack)

ACCURACY

>95%

Robust across diverse demographics

FOOTPRINT

~20 MB

TFLite/ONNX quantized optimized

ANTI-SPOOF

Active

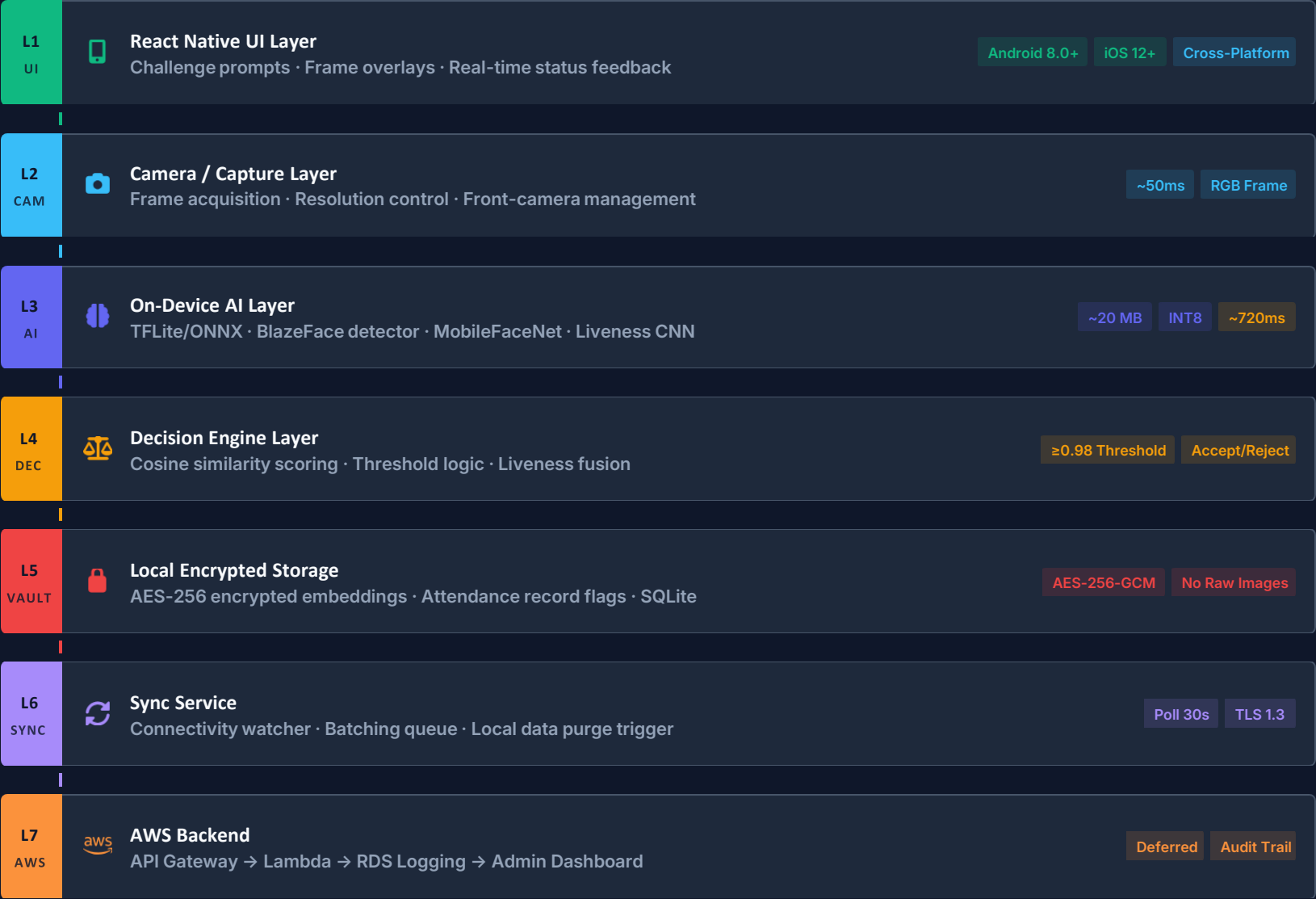
Challenge-response (Blink/Smile)

COMPATIBILITY

iOS/Android

React Native Drop-in Module

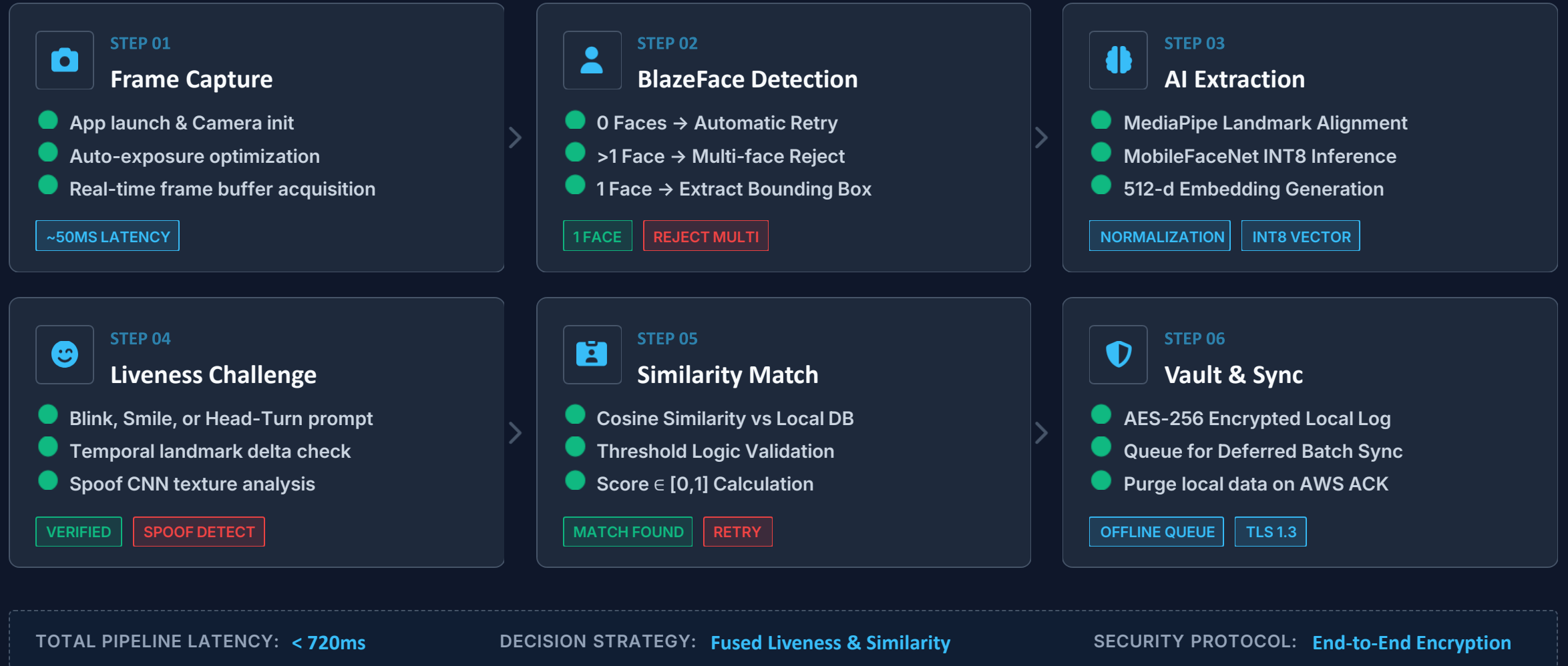
7-Layer System Architecture Diagram



- **Offline Core — L1 to L5** **ALWAYS ACTIVE**
- 100% on-device inference — zero network dependency
 - Privacy-first local processing — raw images discarded
 - Fully operational in mining, rural & disaster zones
 - Hardware-backed AES-256 vault via Android Keystore / iOS Keychain

- **Cloud Sync — L6 to L7** **DEFERRED ACTION**
- Activates only when connectivity is detected
 - Batched encrypted uploads via TLS 1.3 tunnel
 - Purge-on-sync — no persistent raw data retained
- App Edge → API Gateway → Lambda → RDS Log → Dashboard

End-to-End Operational Pipeline Flow



Offline Liveness Detection: Anti-Spoofing Defense

MULTI-LAYERED DEFENSE SYSTEM

LAYER 1 Active Challenges (User Interaction)

-  **Blink Detection:** Track eye aspect ratio
 - Require 1-2 natural blinks in 3-5 sec
-  **Smile Detection:** Lip corner movement analysis
 - Detect genuine smile (DLib + SVM)
-  **Head Pose Movement:** Yaw/pitch/roll estimation
 - Request slight turn (~15°)

LAYER 2 Passive Analysis (No User Action)

-  **Micro-motion Analysis:** Subtle face movements
 - Breathing patterns detection
-  **Texture Analysis:** Screen/print artifacts
 - Moiré patterns, reflection, distortion
-  **Depth Estimation:** Single-image depth cues
 - Dual-camera disparity if available

LAYER 3 Lightweight Spoof Classifier

-  **Binary CNN Classifier:** Real vs Fake
 - Trained on photos, screens, masks
-  **Input:** Face crop + context region
 - Output: Spoof probability score

ATTACK TYPES BLOCKED

-  **Photo Attack**
Printed photo held to camera
-  **Screen Replay**
Video on another device
-  **Mask Attack**
Paper/plastic masks
-  **Cut-photo**
Photo with cutouts

CHALLENGE-RESPONSE PROTOCOL

-  App prompts random challenge →
-  User performs action (blink/smile/turn) →
-  System verifies motion consistency →
-  Classifies REAL vs FAKE →
-   Allow or Block authentication →



Model Compression & Optimization Strategy



OPTIMIZATION TECHNIQUES APPLIED

1



Compact Backbone Selection

- Basis: MobileNetV2 ($\alpha = 0.35/0.50$)
- Fewer layers & channels than full models
- Designed specifically for mobile inference

2

1010
0101

Quantization (INT8 / FP16)

- Converts 32-bit float $\rightarrow$ 8-bit integers
- 4x size reduction, minimal accuracy loss
- HW acceleration: NNAPI, CoreML support

3



Pruning (Structured)

- Removes redundant neurons/connections
- Sparsity-aware training for efficiency
- Further reduction: 30–50%

4



Knowledge Distillation

- Train small “student” to mimic large “teacher”
- Transfer learned representations efficiently
- Maintain >97% of teacher accuracy

5



Input Resolution Reduction

- Standard $224 \times 224 \rightarrow$ Optimized 160×112
- Reduces compute by $\sim 4\times$ while maintaining quality
- Balanced compression by $\sim 4\times$



BEFORE VS AFTER OPTIMIZATION



BEFORE

Large Dense Network
30 MB+ Size • Heavy



AFTER

Compact Optimized
~17 MB • Fast • Light



Performance Metrics Comparison

METRIC	BEFORE	AFTER	
Model Size	~80+ MB	~13–18 MB	~78%
Inference Time	~2.3–5.0s	~600 ms	~74%
Peak Memory	~350+ MB	~80 MB	~77%
Accuracy	93–95%	96–97%	+1–2%



TOTAL
TARGET:

≤ 20
MB

Detection + Recognition + Liveness
Combined



React Native



Model Opt.



Lightweight

AI Model Stack – Face Recognition Pipeline

Component	Model	Size	Precision
Face Detector	BlazeFace	~1.5 MB	INT8
Face Aligner	MediaPipe	~1.0 MB	FP16
Feature Extractor	MobileFaceNet	~12.0 MB	INT8
Liveness CNN	Custom CNN	~3.0 MB	INT8
Total Combined Stack		~17.5 MB	Optimized

- 01

Face Detection & Filtering
BlazeFace locates bounding boxes and landmarks; discards multi-face frames.
- 02

Alignment & Normalization
5-point landmark correction and affine transform for 112×112 face crop.
- 03

Feature Embedding Generation
MobileFaceNet extracts 128/512-d mathematical vector representations.
- 04

Similarity & Match Engine
Cosine similarity computation vs. local templates with INT8 precision.
- 05

Final Decision Logic
Threshold-based validation and AES-256 encrypted local logging.



Privacy-First Architecture

No Raw Image Storage

One-Way Hashing

Encrypted Vectors

On-Device Only

Raw facial images are discarded immediately after Stage 2; only mathematical embeddings persist.

Layered Liveness Detection System

Layer	Method	Technique	Attack Blocked
L1	Blink Detection	Eye-aspect-ratio (EAR) tracking over temporal frames	Static Photo
L2	Smile Detection	Dynamic mouth landmark displacement analysis	Printed Image
L3	Head-Pose	Yaw/Pitch challenge-response (Turn Left/Right)	Screen Replay
L4	Depth Analysis	Monocular depth estimation for 3D liveness	Flat Mask
L5	Spoof CNN	Lightweight CNN for print/screen texture cues	All 2D Attacks

Challenge-Response Engine

Randomized Challenge

"Blink" / "Smile" / "Turn Left"



User Execution

2-3s Temporal Window



Sync Verification

Landmark Delta Analysis



Fused Liveness Score

Threshold > 0.98 Verification

Attacks Mitigated

✓ Photo Attack

✓ Video Replay

✓ Printed Image

✓ Static Mask

Model Optimization & Performance KPIs

 Edge Optimization Stack

TECHNIQUE	METHODOLOGY	IMPACT
Quantization	FP32 to INT8/FP16 weights conversion	4x Size Reduction
Structural Pruning	Removal of redundant neural channels	30% Faster Inference
Distillation	Teacher-Student: Heavy to MobileFaceNet	70% Smaller Model
Model Fusion	BN + Conv layer merging at inference	10% Latency Gain
Runtime Export	Hardware-optimized TFLite/ONNX formats	20% Throughput

 Performance Benchmarks

METRIC	TARGET VS ACHIEVED	STATUS
 Total Latency	<1000ms ~720ms	
 Model Size	<20 MB ~17.4 MB	
 Accuracy	>95% 96.8% HIGH	
 RAM / Power	<3GB 450MB / 1.5%	
 Liveness Score	>0.98 0.982	



Device Compatibility & Runtime

- Minimum: Android 8.0+ / iOS 12+ | 3GB RAM | Snapdragon 600
- Tested: Redmi Note 10, Samsung A52, iPhone SE 2020
- Engine: CPU-only inference via TFLite NNAPI / CoreML

Security & Encrypted Storage Architecture



Encrypted Local Vault

- AES-256-GCM:** Gold standard symmetric encryption with authenticated data integrity.
- Hardware Keystore:** Keys managed in TEE (Trusted Execution Environment) via Android Keystore / iOS Keychain.
- Zero Raw Storage:** No facial images are ever written to disk; system operates solely on mathematical vectors.

```
ENCRYPTED DATA SCHEMA (SQLITE){
  "user_id": "UUID",
  "embedding": "float[512] (AES-GCM)",
  "timestamp": "ISO8601",
  "status": "PENDING_SYNC",
  "session_id": "AUTH_HASH"
}
```

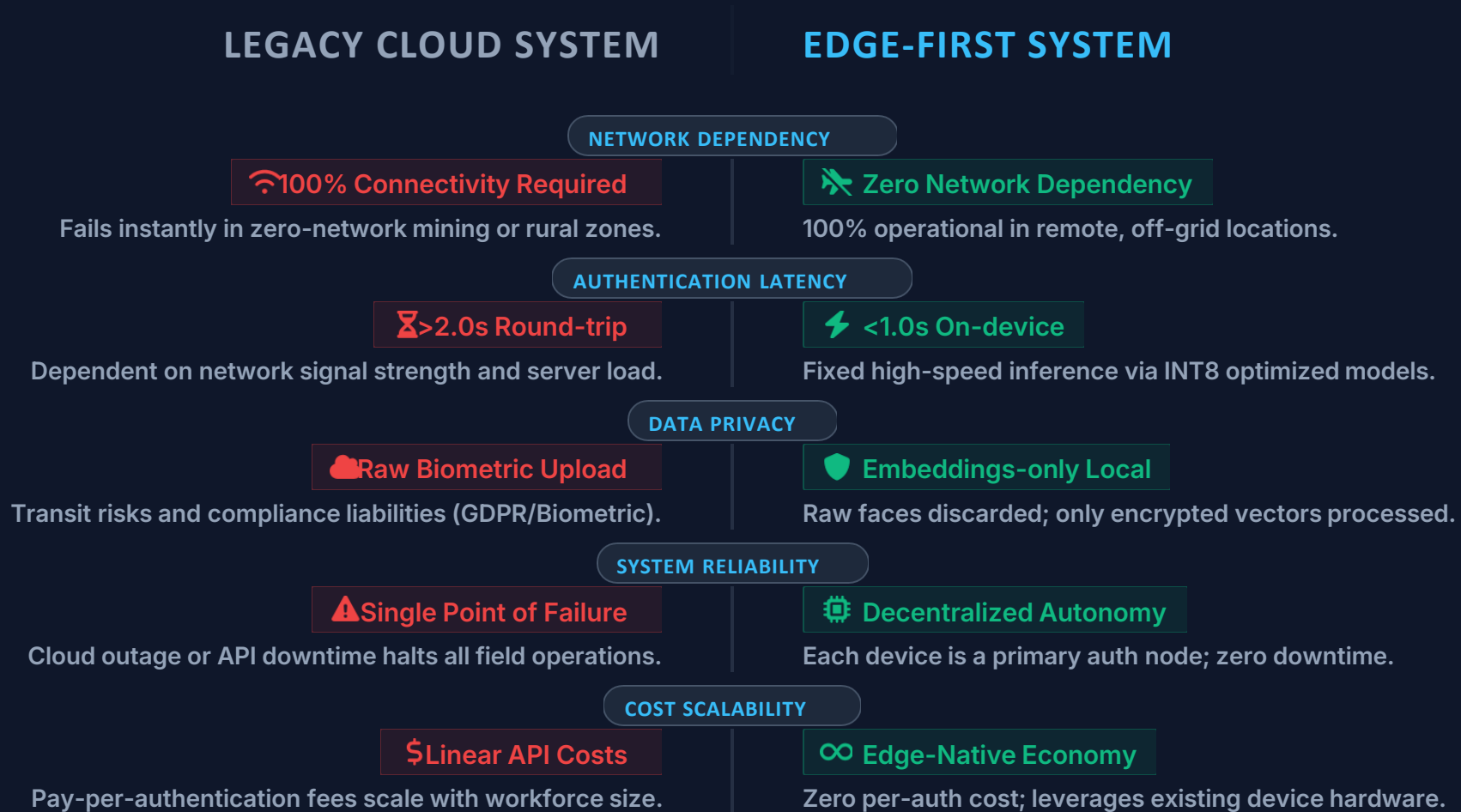
THREAT FACTOR	MITIGATION STRATEGY	CONFIDENCE
Physical Device Theft	AES-256 + Hardware Keystore; embeddings are non-reversible.	High
Biometric Replay	Liveness challenge (blink/smile) prevents static image attacks.	High
Man-in-the-Middle	TLS 1.3 encrypted tunnel during deferred synchronization.	High
Vector Reconstruction	One-way mathematical hashing; no inverse function exists.	Very High
Data Exfiltration	Purge-on-sync protocol; no persistent raw data on device.	High

No Raw Image Storage

Encrypted Vectors

One-Way Hashing

Edge vs Legacy Cloud Comparison



HYBRID SYNC PIPELINE

AWS Deferred Sync: While authentication is 100% offline, the system utilizes a background TLS 1.3 tunnel to batch-upload encrypted audit logs only when connectivity is detected, ensuring centralized visibility without operational dependency.

Use Cases & Real-World Impact

Industrial Deployment



Remote Mining & Infrastructure

Secure authentication in deep mines with zero connectivity.
Ensures worker safety compliance at hundreds of sites.



Construction & Field Operations

Eliminates proxy attendance through on-device liveness checks.
Significant financial fraud reduction for millions of workers.



Rural Workforce Verification

Last-mile biometric trust for NHAI/MNREGA government operations. High robustness across diverse Indian demographics.



Border & Military Posts

Secure personnel authentication in off-grid critical infrastructure where cloud dependency is a security risk.



Datalake 3.0 Integration

```
npm install @datalake/offline-faceid
```

MODULAR DROP-IN

React Native module ready for immediate deployment into existing Datalake 3.0 apps.

API COMPATIBILITY

Fully compatible with existing backend API contracts and logging standards.

ZERO REFACTORING

No changes required to existing app architecture or business logic flow.

NATIVE DELEGATES

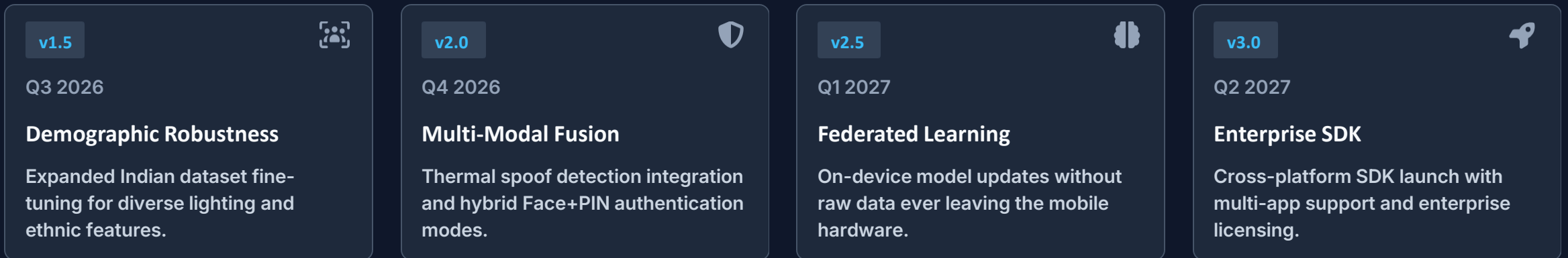
Uses TFLite NNAPI and iOS CoreML for optimized hardware performance.



The system bridges the "Connectivity Gap" while maintaining the strict data integrity standards of the NHAI Datalake ecosystem.

Future Roadmap & Conclusion

Strategic Evolution Phases



Key Value Propositions

